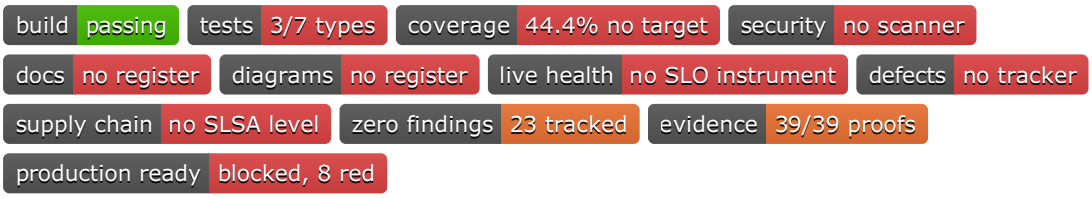
README

vasic



Umbrella monorepo for two production websites and the tooling that generates, translates, validates and ships them. Nothing here is a framework or a product. The sites themselves are git submodules; everything that renders or checks them lives at the top level of this repository.

Two facts to read before anything else:

- **This repository is public. Three of its submodules are not.** Referring to a private path is fine. Copying text out of one into any file here is a permanent public disclosure, because a push cannot be taken back. See Content boundary.
- **There is no server-side CI at this root.** Gate enforcement is a local pre-push hook, and `.git/hooks/` is untracked, so a fresh clone enforces nothing until you install it. See Gates.

Contents

1. What is in here
2. Getting started
3. Building the sites
4. Gates and how to run them
5. The instruments
6. Deploys
7. The submodule fleet

8. Content boundary
9. Governance
10. What is not done
11. Where to go next

What is in here

Path	What it is
`vasic.digital/`	Site submodule. Committed static HTML, served exactly as committed — no build step.
`milosvasic.ru/`	Site submodule. Jekyll source; the rendered `_site/` is git-ignored and must be built before the browser suite can serve it.
`_tools/gen/`	The Go generator (`module vasic.digital/tools/gen`, `go 1.26`). Renders localized pages for both sites from `_content*/` plus `design-system/`.
`_tools/`	The rest of the tooling: `translate/`, `pdf/`, `portfolio/`, `od/`, `deploy-langs.sh`, `audit-hardcoding.sh`.
`design-system/`	Per-brand tokens, fonts, icons, motion and component CSS shared by both sites. One directory per brand plus shared assets.
`_content/`	English source content: `docs/`, `portfolio/`, `products/`, `sites/`.
`_content_<lang>/`	Per-language siblings of `_content/`, one directory per translated language.
`_tests/`	Playwright suite plus the self-validating harness (visual oracle, PDF/OCR export checks, link and sitemap integrity).
`scripts/`	

Path	What it is
	Governance and adaptability instruments, the pre-push gate runner, and the check registry.
`tests/`	The constitution-inheritance test, kept separate from the browser suite.
`docs/`	Per-area documentation: constitution adoption, content boundary, environment adaptability, site health, wizard.
`specs/`	SpecKit feature directories (`spec.md`, `plan.md`, `tasks.md`, contracts, checklists).
`.specify/`	SpecKit itself: templates, workflows, scripts and the vendored extension copy the workflows use.
`_analysis/`	Research and inventory notes that fed design and localization decisions. Working material, not a contract.
`data/`	Small data artifacts used by the tooling.
`submodules/`	Grouped submodules: the governance source, reusable libraries, and one third-party dependency.
`upstreams/`	Per-provider push recipes read by the commit wrapper.
`helix-deps.yaml`	Machine-readable submodule manifest. Every entry pins a commit that must equal the live gitlink.
`CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md`	The four per-agent governance carriers. Identical from line 19 down, and enforced as such.
`Constitution.md`	This project's own constitution. It *extends* the inherited one and overrides no clause of it.

Path	What it is
`CONTINUATION.md`	The resumption contract. Read it first when picking work up.
`.content-boundary-allow`, `.hardcoded-paths-allow`	Declared exemptions for two of the instruments. Each entry needs a stated reason; each is a debt, not a fix.
`.github/workflows/ci.yml.disabled`	The gate definitions, preserved but inert. A provider only reacts to its exact trigger filename, so this file starts nothing.

Re-derive the language list rather than reading one:

```
ls -1d _content_*/ | sed 's#_content_##; s#/##'
```

Getting started

1. Clone and initialise the two site submodules

Initialise them **non-recursively**. `milosvasic.ru` carries a nested gitlink whose own `.gitmodules` is a broken zero-byte entry, so a recursive checkout of the umbrella fails. That nested tree is excluded from the Jekyll build and no gate reads it.

```
git clone git@github.com:milos85vasic/vasic.git
cd vasic
git submodule update --init vasic.digital milosvasic.ru
```

Governance work needs one more:

```
git submodule update --init submodules/constitution
```

2. Toolchains

Tool	Used by
Go 1.26	`_tools/gen` unit tests, the generator build, the hardcoding audit
Node 20	the `_tests` Playwright suite and the portfolio validators

Tool	Used by
Ruby 3.3 + Bundler	the `milosvasic.ru` Jekyll build
`poppler-utils`, `tesseract-ocr` + language packs	the export self-validation (`pdftotext`, `pdfimages`, `pdftoppm`, OCR)
WebKit system libraries (`libmanette`, `libwoff2`)	the Playwright `webkit` project
`python3` with `PyYAML`	manifest parsing in several instruments

Without poppler and tesseract the export check reports a stated SKIP instead of running. A SKIP is printed loudly and is not a pass.

Host packages are installed automatically

You do not install the host packages above by hand. `scripts/hostdeps.sh` installs them. The setup wizard (Step 10) installs all of them. Pre-push gate 5 installs tesseract with its language packs and poppler (`--only ocr --only pdf`). Gate 6 only downloads the Playwright browser; when a browser cannot start, its preflight prints the `--only webkit` command that installs the WebKit libraries.

```
bash scripts/hostdeps.sh check                    # report only;
        one machine-readable line per package
bash scripts/hostdeps.sh install                  # install
        whatever is missing, in one package-manager call
bash scripts/hostdeps.sh install --dry-run
        # show the plan and the exact command; change nothing
bash scripts/hostdeps.sh install --print-commands  # print only
        the exact command for this host
bash scripts/hostdeps.sh install --only webkit
        # capabilities: webkit, ocr, pdf
```

It covers the WebKit libraries, `tesseract` with the `eng osd srp srp_latn rus deu fra` language packs, and poppler. It picks the package names for your distribution: apt-get on Debian/Ubuntu, apt-get on ALT Linux, dnf, yum, zypper, pacman, apk, Homebrew and MacPorts. Installing the language packs does not turn OCR on; per-book OCR stays opt-in and off by default.

It never asks for a password. It installs only when it runs as root or when `sudo -n` works without one. Otherwise it prints the exact command for your host and exits **2**, and an exit 2 is not a pass. Run that command once as an administrator, then run `check`. Some packages have no name recorded for some distributions: the Alpine `srp_latn` pack, and poppler on ALT Linux and MacPorts. For those it also exits **2**

and names what is missing rather than guess a package name. On Alpine the WebKit libraries install and are found by path, because musl has no `ldconfig -p`, but Playwright's WebKit build needs glibc and still cannot run there. `HOSTDEPS_AUTO_INSTALL=0` turns automatic installs off; `--yes` overrides that for a single run.

3. Verify the toolchain answers

```
go version && node --version && ruby --version && bundle --version
command -v pdftotext pdfimages pdftoppm tesseract
bash scripts/hostdeps.sh check
python3 -c 'import yaml; print(yaml.__version__)'
```

Building the sites

The generator reads `_content*/` and `design-system/` and writes into the two site submodule directories. No page is hardcoded.

```
# Build the generator and render both sites for English, then
        rebuild _site:
bash _tools/gen/build.sh

# Render one language:
bash _tools/gen/build.sh --lang de

# Render into a scratch directory instead of the live site dirs:
bash _tools/gen/build.sh --out /tmp/render --no-jekyll
```

The binary underneath takes explicit flags, which is what `build.sh` drives:

```
cd _tools/gen && go build -o gen . && ./gen -h
#   -site   vasic.digital | milosvasic.ru
#   -lang   language code (default en)
#   -what   all | products | portfolio | home
#   -root   repository root
#   -out    output base dir
```

`milosvasic.ru` additionally needs its Jekyll build before anything can serve it:

```
cd milosvasic.ru && bundle install && \
  JEKYLL_ENV=production bundle exec jekyll build --destination
        _site
```

Gates and how to run them

There is no CI at this root

The workflow that used to run on push is renamed to `ci.yml.disabled` and does nothing. The same gate set is enforced by a local hook. Three consequences, stated plainly:

- A fresh clone has **no** enforcement until the hook is installed.
- `git push --no-verify` skips the hook and leaves no record.
- Nothing on the server re-checks a push that skipped it.

Confirm the root is free of an active tracked CI config — this must print nothing:

```
git ls-files | grep -E '^\.github/workflows/.*\.ya?ml$|^\.gitlab-ci\.yml$'
```

Install the hook, once per clone

The logic is tracked in `scripts/pre-push-gates.sh`; the hook itself is a small shim that runs it, so editing the script takes effect with no reinstall.

```
bash scripts/pre-push-gates.sh --install    # write .git/hooks/pre-push
bash scripts/pre-push-gates.sh --list       # print the gate table, run nothing
bash scripts/pre-push-gates.sh              # run them now
bash scripts/pre-push-gates.sh --uninstall  # remove the shim
```

`--install` is idempotent and takes a hardlinked backup of any pre-existing `pre-push` hook before writing over it.

The gate table

Print it from the runner rather than trusting a copy:

```
bash scripts/pre-push-gates.sh --list
```

Measured 2026-09-03, that runner declares eight gates, `E` and `0` through `6`: the no-active-root-CI self-check, the hardcoded-path audit, the Go unit tests, the hardcoding audit, the translation reproducibility self-test, the portfolio data-integrity self-validation, the harness self-validation, and the Playwright chromium suite with the live all-language crawl excluded.

Each one, runnable on its own:

```
bash scripts/audit-hardcoded-paths.sh
cd _tools/gen && go test ./... ; cd -
bash _tools/audit-hardcoding.sh
bash _tools/translate/reproducibility-selftest.sh
bash _tools/portfolio/self-validate.sh
bash _tests/run-harness-selfvalidation.sh
```

Four environment switches change how the runner behaves:

Variable	Effect
`PREPUSH_VERBOSE=1`	Stream each gate's output live instead of only on failure
`PREPUSH_SKIP_SLOW=1`	Skip the full Playwright gate, with a printed reason
`PREPUSH_STRICT=1`	Treat every SKIP as a failure. Use before a release
`PREPUSH_ONLY="E 0 1"`	Run only the listed gate ids

Gates 5 and 6 SKIP with a stated reason when `_tests/node_modules` or the built `milosvasic.ru/_site` is missing. A SKIP is reported loudly and is not a pass — `PREPUSH_STRICT=1` is how you refuse to accept one.

There is deliberately **no** switch that skips gate E. The rule it enforces names and refuses the whole family of opt-out flags by name, so adding one would be a violation rather than a convenience.

The browser suite

```
cd _tests
npm ci
npx playwright install chromium
npx playwright test --project=chromium --grep-invert "all-
        language"
```

Both sites are served locally by the harness. Ports and base URLs come from one tracked source, `_tests/env.js`, so a port and the base URL that addresses it cannot drift apart. Every value is environment-derived with the previous literal kept as the documented default:

```
VD_PORT=9401 MV_PORT=9082 npx playwright test --project=chromium
```

An unparseable port throws when the module loads instead of silently falling back. Read `_tests/env.js` for the full variable list.

Which specs do not run in the push gate

`_tests/playwright.config.js` carries a `testIgnore` that excludes three specs, and the local run above excludes a fourth by `--grep-invert`. All four assert against live production, and all four are claimed by `_tests/playwright.live.config.js` instead:

```
grep -n 'testIgnore' _tests/playwright.config.js
grep -n 'testMatch'  _tests/playwright.live.config.js
```

The reason for the split is measured, not stylistic: gating a push of undeployed source on public DNS makes the gate fail for reasons that have nothing to do with the change being pushed. The question those specs answer is whether what shipped is correct, so they run after a deploy, against what shipped. `_tools/deploy-langs.sh` names all four and runs them there.

Run them by hand before a release:

```
cd _tests && npx playwright test --
        config=playwright.live.config.js
```

The instruments

Separate from the push gates. These judge the repository itself — its governance wiring, its manifests, its adaptability, and its boundary with the private submodules.

Every one of them is three-valued.

Exit	Meaning
`0`	nothing found
`1`	a real finding, in the tree
`2`	no verdict was reached

An exit of `2` is not a pass. It reports that the instrument could not look at what it judges — an uninitialised submodule, no provider client, no network, a missing text extractor — and calling that outcome clean would be a bluff. Treat it as work.

Script	What it judges
`scripts/continuation-check.sh`	`CONTINUATION.md` is not stale: entry points it names exist and

Script	What it judges
	are executable, gap statuses agree with the carriers, and no commit changed a watched governance file without updating it
`scripts/verify-governance-cascade.sh`	The whole governance wiring in one pass: fleet classification from evidence, carrier presence, pointer-inheritance form, manifest/fleet agreement in both directions, nested gitlinks, per-submodule carrier lockstep, and manifest pins
`scripts/verify-manifest-pins.sh`	Standalone form of the pin check: every `helix-deps.yaml` ref equals the gitlink this repository would commit
`scripts/verify-submodule-remote-sync.sh`	Every declared gitlink against its remote. The only instrument here that looks at a remote at all
`scripts/verify-check-registry.sh`	The registry itself: every registered check points at a real, executable file, and every script under a declared scan root is registered
`scripts/verify-content-boundary.sh`	Private prose appearing in a public repository of the fleet
`scripts/verify-private-object-exposure.sh`	Private git objects reachable from a public repository
`scripts/audit-hardcoded-paths.sh`	Machine-specific absolute paths
`scripts/audit-environment-assumptions.sh`	Frozen hosts, ports, model names, tool versions and GNU-only syntax with no override layer
`scripts/verify-all-constitution-rules.sh`	The full rule sweep. Discovers its gates at run time
`scripts/verify-mutation-anchors.sh`	Paired-mutation proofs that have rotted away from what they claim to guard

Script	What it judges
`scripts/verify-remedy-executability.sh`	Remedies printed by other instruments are actually runnable
`scripts/verify-name-in-path.sh`	Personal names appearing in tracked paths
`scripts/verify-provider-ci.sh`	What no file-level check can see: Pages build type, provider-generated deployment runs, branch protection, Actions enablement
`scripts/lumen-index-doctor.sh`	Health of the local semantic index
`scripts/ollama-tune.sh`	Concurrency for the local inference host, computed from measured CPU and RAM

Run any of them directly. None of them changes the repository. Two touch state outside it and only when told to: `ollama-tune.sh` is report-only unless `--apply` is passed, and `lumen-index-doctor.sh` inspects an index it does not rebuild.

Each registered check also ships a paired proof that it fails when the condition it guards is broken, and a distinct path that returns “could not determine”. A gate with no failing case has never been shown to catch anything:

```
bash scripts/verify-governance-cascade.sh --prove-failure
bash scripts/verify-content-boundary.sh  --prove-failure
```

Seeing where the tree stands right now

Nothing in this file is a substitute for running them. This sequence is the short version, and it takes minutes rather than seconds — the boundary scan and the rule sweep are both slow:

```
bash scripts/continuation-check.sh          # is the handoff
        document stale?
bash scripts/verify-governance-cascade.sh   # is the governance
        wiring intact?
bash scripts/verify-check-registry.sh       # is every check
        accounted for?
bash scripts/verify-submodule-remote-sync.sh
```

```
bash scripts/audit-environment-assumptions.sh
bash scripts/verify-content-boundary.sh     # expect 1; read it,
        do not silence it
```

Record what each one printed, not what this file says it prints.

The check registry

`scripts/check-registry.tsv` is the enumeration of "every check". It exists because two success criteria in `specs/001-…/spec.md` are quantified over that set, and a claim quantified over a set that does not exist cannot be measured.

It closes the enumeration in both directions. Every row must point at a real file, and every `*.sh` under a declared scan root must appear in some row — as a check, a declared debt, or an exemption with a stated reason. **Adding a script under `scripts/` therefore fails the registry until it is registered.** That is the rule working, not a defect.

```
bash scripts/verify-check-registry.sh              # structure
        only
bash scripts/verify-check-registry.sh --run-proofs # actually
        execute the paired proofs
bash scripts/verify-check-registry.sh --strict
        # make declared debt block
```

A plain run verifies that each check *declares* a paired failure proof and a distinct could-not-determine path. It does not run them. `--run-proofs` does. The distinction matters: a proof that returns success while exercising nothing is the failure mode the registry was written to catch.

Reference: docs/check-registry.md.

Two of these are expected to be red (one conditionally)

- **`verify-content-boundary.sh` is designed to stay red: exit `1` = findings present** (re-measure: `bash scripts/verify-content-boundary.sh; echo "rc=$?"`). See Content boundary. A finding there is a reading assignment. Do not silence it, do not allow-list your way to green.
- **`verify-submodule-remote-sync.sh` is conditional, by exit code:** exit `0` = every owned gitlink is current, exit `1` = at least one pin is behind its remote or differs from it, exit `2` = it could not determine (never a pass) (re-measure: `bash scripts/verify-submodule-remote-sync.sh; echo "rc=$?"`). Bumping a pin is an operator decision, and this instrument does

not make one, so while a pin is behind it stays at `1` until an operator acts.

Neither is a reason to weaken the instrument.

Deploys

```
bash _tools/deploy-langs.sh --dry-run   # preview: build, no
        commit, no push
bash _tools/deploy-langs.sh             # the real cycle
```

One cycle regenerates English plus every **complete** language into both site directories, commits and pushes each site only where something actually changed, then validates the live sites. A language counts as complete when every source document carries a passing review verdict.

Its exit contract distinguishes five outcomes, and the distinctions are load-bearing:

Exit	Meaning
`0`	deployed; live validation passed or was explicitly skipped
`1`	deployed, and the live suite reached production and found a real defect there — act on the sites
`2`	deployed, but the live suite reached no verdict at all — act on the toolchain or the network path, then re-validate
`3`	refused to deploy: unrelated changes were present in a site submodule. Nothing was built, staged, committed or pushed
`4`	the publish itself failed. Live validation is deliberately skipped, because crawling live would report on content this run did not publish

`milosvasic.ru` publishes itself, and that must not change

`milosvasic.ru/.github/workflows/pages.yml` is active, and it is the only path by which that site is published. Its repository root is Jekyll source — Liquid and front matter — so it cannot be served raw from a branch, and there is no alternative publishing branch or folder.

`_tools/deploy-langs.sh` is not a substitute. It generates, commits and pushes *source*, then waits for the server to rebuild. The rebuild it waits for **is** that workflow. Disabling it would not downgrade publishing to manual; it would stop the site updating at all.

Do not disable, rename, or otherwise “fix” that file.

`vasic.digital` needs no build step, but its Pages source setting still makes the provider run a deployment on every push, with zero workflow files in its tree. Nothing inside that repository can change this.

Every provider setting named here lives outside this tree and can change without leaving a trace in it. Measure instead of quoting:

```
bash scripts/verify-provider-ci.sh
#   0 = no provider-generated triggering found
#   1 = provider-side triggering confirmed (operator-only to
        change, in the provider UI)
#   2 = could not determine — not a pass
```

It enumerates the owned repositories from this checkout’s own remotes and asks each provider directly. Historical reading, 2026-09-03 (re-measure with `bash scripts/verify-provider-ci.sh; echo $?`): it exited `1`, with the confirmed standing trigger being the `vasic.digital` Pages source setting, and six rows reported as unverified because no read-only API adapter is registered for those hosts. Note the shape of that output: a confirmed finding does not suppress the unverified rows. They are printed on every run, so a `1` never hides them and they never read as clean.

The submodule fleet

The fleet is **derived**, never hardcoded. Two files define it, and they are checked against each other in both directions:

```
git config -f .gitmodules --get-regexp 'submodule\..*\.path'   #
        the gitlinks
python3 -c 'import yaml;d=yaml.safe_load(open("helix-
        deps.yaml"));print(len(d["deps"]))'
bash scripts/verify-governance-cascade.sh    # classifies the
        fleet from evidence
bash scripts/verify-manifest-pins.sh         # every recorded ref
        vs its live gitlink
```

Measured 2026-09-03 by exactly those commands: **13 gitlinks are declared, 12 carry a manifest entry, and the cascade classifies 11 as owned consumers, 1 as the governance source, and 1 as third party.** Those numbers move when a submodule is added. The commands above are the authority; this paragraph is a dated observation.

Three roles exist:

- **Owned consumer** — this project owns it and the governance cascade reaches it.
- **Governance source** — the constitution. It is the head of the cascade, not a consumer of it.
- **Third party** — upstream code this project consumes and does not own. It is probed and reported, and it is never an input to any verdict.

Visibility is a provider setting — re-derive it

Visibility is configured outside this repository and changes without any commit here. Do not trust the table below; regenerate it:

```
for u in $(git config -f .gitmodules --get-regexp
        'submodule\..*\.url' | awk '{print $2}'); do
  slug=${u#git@github.com:}; slug=${slug%.git}
  printf '%-45s %s\n' "$slug" "$(gh api "repos/$slug" --
        jq .visibility 2>/dev/null || echo COULD_NOT_DETERMINE)"
done
```

Measured 2026-09-03 with exactly that loop:

Path	Role	Visibility	What it is
`submodules/constitution`	governance source	public	The universal constitution this repository inherits: the corpus, the four per-agent carriers, the resolver script, the propagation and covenant gates, the post-

Path	Role	Visibility	What it is
			pull hook, and the consumer templates
`vasic.digital`	owned	public	Site submodule. Committed static HTML, no build step
`milosvasic.ru`	owned	public	Site submodule. Jekyll source; `_site/` is git-ignored
`design-toolkit`	owned	public	Design-system engine: machine-readable tokens, brand contract, design-library catalogue, and design-specialist agent recipes
`submodules/containers`	owned	public	The mandated container library — runtime auto-detection, endpoint discovery, lifecycle and health, compose orchestration, cross-build, on-demand service boot. Go module `digital.vasic.containers`
`submodules/LLMProvider`	owned	public	Provider abstraction for model backends: one interface over many concrete adapters. Go module `digital.vasic.llmprovider`. Adopted rather than rebuilt
`submodules/RAG`	owned	public	Retrieval library: one document identity type and a one-method retriever interface. Go module `digital.vasic.rag`. Adopted rather than rebuilt
`submodules/verdict`	owned	public	The three-valued instrument result whose numeric value *is* the

Path	Role	Visibility	What it is
			process exit code, so exiting on it is correct by construction
`submodules/passage`	owned	public	Stable identity for citable spans of text, minted once at ingest and derived from neither content nor position
`ai_interviewing`	owned	**private**	Interview-preparation corpus and employer due-diligence material
`monetization`	owned	**private**	Monetization documentation and the per-provider upstream declaration scripts
`workshop`	owned	**private**	Teaching-curriculum corpus and the platform built against `specs/001-…`. Recorded sessions and their written materials live under `workshop/chapters/<NN>/`, kept in git as size-capped, checksum-verified archive parts
`submodules/superspec`	third party	public	Upstream spec-kit extension. Not owned; excluded from tagging and propagation

The two reusable libraries `verdict` and `passage` were extracted out of `workshop` and published as their own public repositories precisely because a submodule nested inside a submodule is forbidden, and because a general-purpose library has no business living inside a private corpus.

Nested gitlinks

`milosvasic.ru` contains one nested gitlink in a namespace this project does not own. It is classified from evidence, documented in the manifest, and excluded from the cascade — which is also why the clone step above is non-recursive.

The `design-toolkit` mirrors are not in sync

That submodule has two remotes declared in this checkout, and they hold different heads. Re-derive rather than quoting a number:

```
git -C design-toolkit remote -v
git -C design-toolkit ls-remote origin HEAD
git -C design-toolkit ls-remote gitlab HEAD
```

Measured 2026-09-03: the GitLab head is a strict ancestor of the GitHub head — six commits behind, zero divergent. That is lag, not divergence, and the remote-sync instrument reports this submodule current because it probes the declared origin only. “Current” there is a statement about one remote.

Content boundary

This repository is public. `workshop`, `ai_interviewing` and `monetization` are private. A gitlink publishes a commit id and nothing else, so their contents stay private — right up until somebody copies a paragraph out.

The rule is a single line:

> Naming a private path is allowed. Copying what is inside it is not.

Written into a public file here	Verdict
A path under a private submodule, on its own	allowed
A filename, a directory, a count of files	allowed
A sentence, a heading, a table row, a schema header lifted from inside one	forbidden
	forbidden

Written into a public file here	Verdict
A person's real name carried only by private material	

Nobody does this by being sloppy. It happens while writing documentation *well*: you want an example that makes the point concrete, and the most concrete example to hand is the real one. That example is the disclosure, and a single paragraph of it stays public for good, because a push cannot be edited out of history.

`workshop/chapters/**` holds a recording of a private teaching session with an identifiable third party. **Refer to it by path. That person's name belongs in no file in this repository.**

The instrument

```
bash scripts/verify-content-boundary.sh
bash scripts/verify-content-boundary.sh --list-fleet          #
        the derived public/private map
bash scripts/verify-content-boundary.sh --include-untracked   #
        also read untracked public files
```

It shingles the private side, subtracts what is demonstrably already public, and reports what survives with both sides named. Legitimate cross-boundary flows are declared as private/public **pairs** with a mandatory reason in `.content-boundary-allow`.

It is designed to stay red, exit `1` = findings present (re-measure: `bash scripts/verify-content-boundary.sh; echo "rc=$?"`). Do not make it green. The surviving matches are left visible rather than allow-listed, because an allow-list entry buys a green exit at the price of hiding the row from the next reader.

Three things about that number, stated so it is not misread:

1. **It detects co-occurrence, not direction.** Governance text that originated in this repository's own carriers and propagated *into* a private submodule is, to this instrument, indistinguishable from private text leaking out.
2. **The count moves when the private side grows**, with no change to the instrument and no new disclosure. Compare counts only against a run of the same day.
3. **The rows have not been individually judged.** An assessment covering a few hundred matches exists; the reported population is far larger. Do not read any figure from it as "cleared".

Full design and the fleet map: `docs/content-boundary.md`.

Untracked files are invisible to a default run

The scan enumerates tracked files. An untracked public file quoting private material is not seen unless `--include-untracked` is passed — and the commit wrapper used in this project stages everything untracked. Pass the flag before committing new files.

Governance

Where the rules live

The authority is the git submodule at `submodules/constitution/`. Nothing is copied out of it into this repository. Note the path: the constitution's own prose uses a top-level `constitution/` in its examples; here it is `submodules/constitution/`. The bundled resolver finds it from any depth:

```
bash submodules/constitution/find_constitution.sh
```

What	Path
The corpus	`submodules/constitution/Constitution.md`
Claude Code carrier	`submodules/constitution/CLAUDE.md`
Codex / Cursor / Aider / OpenCode / Crush carrier	`submodules/constitution/AGENTS.md`
Qwen Code carrier	`submodules/constitution/QWEN.md`
Gemini CLI carrier	`submodules/constitution/GEMINI.md`
Parent-walk resolver	`submodules/constitution/find_constitution.sh`
Post-pull hook	`submodules/constitution/scripts/post_update_hook.sh`
Forbidden-command guard	`submodules/constitution/scripts/hooks/guard-forbidden-commands.sh`
Propagation and covenant gates	

What	Path
	`submodules/constitution/`
	`scripts/gates/`

Read one anchor rather than the whole corpus:

```
awk '/^### §11.4.35 /{f=1} f&&/^### §11.4.36 /{exit} f{print}' \
    submodules/constitution/Constitution.md
```

The four root carriers

`CLAUDE.md`, `AGENTS.md`, `QWEN.md` and `GEMINI.md` at this root are the same document with four different opening sections. Lines 1 to 18 are per-agent; from line 19 to end of file all four are byte-identical, and a gate enforces exactly that range — it measures where the four actually converge and refuses a declared split narrower than the measured one, so there is no identical-but- ungated remainder.

Verify it yourself; expect `4` then `1`:

```
for N in 18 19; do
  printf 'from %s: ' "$N"
  for f in CLAUDE.md AGENTS.md QWEN.md GEMINI.md; do tail -n +$N
        "$f" | sha256sum; done \
    | sort -u | wc -l
done
```

Edit the shared region as one artifact — split the head off, edit the tail once, recompose. Editing four files by hand and relying on the gate to catch a slip is how they drift.

This project's own constitution

`Constitution.md` at this root is a *consumer* document. It extends the inherited corpus and it overrides nothing in it — a consumer may add rules, and may not weaken one it inherits. The only legitimate way to record an exception is the explicit override form the constitution's own template defines, and it cannot be used against a clause that refuses exemptions by name.

Measured 2026-09-03: no clause of the universal constitution is overridden by this project. Two positions that look like exceptions are **documented deviations**, not overrides — see What is not done.

Pointer inheritance, and why it is used here

A consumer may wire the inheritance in either of two forms, and the rule declares them equivalent: a native import of the carrier, or a portable `## INHERITED FROM …` pointer block as the first heading. This repository uses the pointer.

The reason is cost, and it is stated openly rather than left implicit:

```
du -h submodules/constitution/Constitution.md submodules/
        constitution/CLAUDE.md
```

Measured 2026-09-03: the corpus is 1.7 MB and the carrier 784 KB. A native import pulls the carrier into context before a session does any work at all — on the order of 165 thousand tokens, every session, whether or not a single rule is relevant. The pointer carries the same authority at no standing cost, and it is the form the constitution's own propagation gates recognise: any carrier whose first line-anchored heading outside a code fence is `## INHERITED FROM` is classified as a legitimate pointer consumer.

The trade is real and worth naming: an agent that does not resolve pointers sees less. That is why the carriers restate a short, bounded list of critical rules — a list authored by the constitution for exactly that purpose — and nothing more.

The manifest

`helix-deps.yaml` records every submodule with its URL, its pinned ref, and a stated reason for its presence. A `ref` is a **pin**, not a souvenir: it is the commit the corresponding gitlink points at, and the tree must match it.

Move the gitlink and the manifest entry **in the same change**. Staging one without the other makes the pin check fail, correctly, until both move.

```
bash scripts/verify-manifest-pins.sh
```

The submodule pin is a recurring decision, not a task

`scripts/verify-manifest-pins.sh` compares the manifest to the **local** gitlink. It cannot see a remote, and it never could. A submodule arbitrarily far behind its upstream is invisible to it and to the cascade check that wraps it.

Only one instrument here looks at a remote:

```
bash scripts/verify-submodule-remote-sync.sh
```

Historical reading, 2026-09-03 (re-measure with the command above): it exited 1, the constitution gitlink differing from its remote head, with the *difference* determined and the *direction* not, because the remote commit is not in this checkout's object store. Classifying the direction requires a fetch, and a fetch is a mutating command and an operator decision. The read-only probes tell you *that* it differs, never which way:

```
git -C submodules/constitution rev-parse HEAD
git ls-remote git@github.com:HelixDevelopment/
        HelixConstitution.git HEAD
```

The record in this repository shows that pin brought level with its upstream three times, and it differs again today. **Treat it as a standing operator decision that recurs, not as something that completes.** Do not read any commit id printed in any document here as current — run the check.

The commit wrapper

Use the project's wrapper rather than direct `git add` / `git commit` / `git push` on the main repository. It reads the per-provider recipes in `upstreams/`. It runs `git add .`, which stages everything untracked, so keep `.gitignore` accurate and run the content-boundary scan with `--include-untracked` first.

Never force-push. A force-push needs explicit per-session authorization and a green post-operation gate.

What is not done

This section exists because reporting a state you have not verified is forbidden here, and because a README that oversells is worse than a short one.

The full audit is `docs/constitution-adoption/INVENTORY.md`; its own status ledger is the authority for the identifiers below.

Gap	Status recorded in the inventory
G1, G2, G3, G6, G7	closed

Gap	Status recorded in the inventory
G4	partial, and cannot reach closed from inside this tree
G5	partially closed — the commit wrapper exists; git hooks as a tracked, travelling artifact do not
G8	open — the markdown/export mandate is unmet across the repository, no work in flight
G9, G10, G11	no verified current status. Do not report them either way without re-auditing
G12	open — no `PreToolUse` guard is wired, although the canonical guard script sits unused in the constitution submodule

Re-read the ledger rather than this table:

```
sed -n '/Current status ledger/,/^### G1 /p' docs/constitution-
        adoption/INVENTORY.md
```

Provider-level non-compliance, with no file-level remedy

Disabling a workflow file stops file-triggered runs. It does not reach provider settings — organisation-default required workflows, branch-protection required checks, the Pages source setting, provider-side scheduled exports. Those are operator-only to change, in a provider UI.

Two known positions, both deliberate, neither an override:

- `milosvasic.ru` keeps an active deploy workflow. It is a **documented deviation** taken for production uptime, because that workflow is the only publish path the live site has.
- `vasic.digital` is non-compliant at the provider level and has **no** file-level remedy, because it has zero workflow files.

The rule those two deviate from names and refuses the exemption vocabulary outright, and a consumer may only extend an inherited rule, never weaken it. A project-local override is therefore structurally impossible, not merely disfavoured. **Do not propose one, and do not write either position up as one.**

Record: `docs/constitution-adoption/DECISION-11-4-156-COMPLY.md`. CI surface inventory: `docs/constitution-adoption/CI-INVENTORY-11-4-156.md`.

The content-boundary incident of 2026-09-01 is contained, not remedied

Private material was written into this public repository, committed, and pushed. Three classes crossed the boundary: prose quoted out of a private document, script and schema headers copied out of a private repository, and a third party's real name.

The working tree is redacted. The commits are not. Redaction is containment. Those commits still sit in a public remote, and no push can be un-made. Taking the content out of the record means a rewrite of published history — an operator decision, prepared and not executed.

Do not read the redaction, or any green gate, as closure. The instrument that now watches for a recurrence was written *after* the leak, so it is evidence that the class is watched and evidence of nothing about what is already public.

Forensics, the assessment, and the unexecuted remediation plan: `docs/content-boundary-incident-2026-09-01.md`.

The rule sweep is not green

`scripts/verify-all-constitution-rules.sh` discovers its gates at run time, and the set it discovers changes when the constitution pin moves. A pass/fail split measured before a pin move is therefore not comparable to one measured after it, and the record here contains exactly that discontinuity. **Run it; do not quote a number for it.**

Two classes of failure inside it cannot be cleared by any commit this repository can make, and neither licenses treating the sweep as green:

1. Third-party and vendored carriers. The cascade check reports these as known-unclearable and excludes them from its own verdict; the propagation gates inside the sweep still count them. Excluded there means not double-counted — never suppressed.
2. Defects internal to the constitution submodule, which is upstream code this repository consumes rather than owns. The precedent on

record is that waiting for upstream is the correct response, not committing into a repository this project does not own.

Host capability is measured, never assumed

What can be built here depends on what is installed, and that has changed under this repository more than once. Two traps are worth naming because both have already caused a wrong conclusion:

- A name on `PATH` proves nothing about what that name is. `command -v` finding a binary is not evidence that the binary does what its name suggests.
- A system interpreter and a project virtualenv are different environments. A probe run against the wrong one reports a capability that is present as absent, or the reverse.

Probe before relying on any of it:

```
command -v podman docker
ollama list
```

Where to go next

You want to	Read
Pick up work someone else left	`CONTINUATION.md` — the resumption contract. It carries a verbatim prompt you can paste into any CLI agent, and `bash scripts/continuation-check.sh` tells you whether it is stale before you trust it
Understand the agent rules that bind work here	`CLAUDE.md` — or the carrier matching your agent, `AGENTS.md` / `QWEN.md` / `GEMINI.md`. They are byte-identical from line 19 down
Understand how the constitution is inherited	`docs/constitution-adoption/README.md`, then `INVENTORY.md`
Know what may and may not be written here	`docs/content-boundary.md` and `docs/content-boundary-incident-2026-09-01.md`
Add or change a check	

You want to	Read
	docs/check-registry.md and scripts/check-registry.tsv
Understand the environment-adaptability baseline	docs/environment-adaptability/AUDIT.md
See the live-site health record	docs/site-health/PRODUCTION-SITE-HEALTH.md
Work on a planned feature	specs/<NNN>-<name>/ — spec.md, plan.md, tasks.md, contracts/, checklists/
Know what the setup wizard could not do for you	MANUAL-STEPS.md
Read the shared design tokens and brand contracts	design-system/README.md
Understand the test taxonomy	_tests/TEST-TYPES.md and _tests/GATES.md

Before you claim anything passes

Run it. A gate you have not executed is not a pass, and the absence of a gate is not one either. Every claim in this file either carries the command that produces it or names the document that holds the evidence; keep it that way.